# Отчёт по выполнению задания по курсу «Суперкомпьютерное моделирование и технологии»

Куприй Р.М., 623 группа
Факультет ВМК МГУ имени М.В. Ломоносова

## 1. Постановка задачи

Необходимо решить трёхмерное гиперболическое уравнение в области, представляющей собой прямоугольный параллелепипед. Для поиска решения применяется явная разностная схема 2 порядка точности по времени и расстоянию с заданными начальными и граничными условиями. Тип граничных условий и вид аналитической варианты определяется вариантом задания **№4**: условие 1 рода по ***x*** и периодические условия по ***y*** и ***z***, аналитическая функция:

$$\sin(\frac{3\pi}{L_x}x) \cdot \sin(\frac{2\pi}{L_y}y) \cdot \sin(\frac{2\pi}{L_z}z) \cdot \cos(a_t \cdot t + 4\pi), a_t = \pi\sqrt{\frac{9}{L_x^2} + \frac{4}{L_y^2} + \frac{4}{L_z^2}}$$

Численный метод для вычисления решения основан аппроксимации исходного уравнения на семиточечной разностной схеме и использует значения решения с предыдущего шага для вычисления значений на текущем шаге. Таким образом, такая схема является явной. Для вычисления значений функции на 0 и 1 шагах используется аналитическое значение функции, заданное в программе явно.

Для выполнения задания необходимо написать последовательную программу, распараллелить её с использованием средств OpenMP, MPI и OpenMP + MPI и затем сравнить производительность получившихся реализаций с последовательной версией.

## 2. Программная реализация

Основной вычислительной процесс при решении представляет собой цикл по трём измерениям (x, y, z) на каждом шаге по времени. При этом, зависимостей по данным между итерациями этого цикла нет, что значительно облегчает распараллеливание программы.

При использовании средств OpenMP, были добавлены прагмы **#pragma omp parallel for** для распараллеливания имеющихся циклов: вычисление аналитического решения для шагов 0, 1; основного цикла по трём измерениям и при вычислении погрешности итогового решения. Использование параллельных регионов, а также дополнительных опций к прагме (таких как, например, **collapse**) не приводит к увеличению эффективности.

При использовании средств MPI необходимо обеспечить блочное 3d разбиение расчётной области между процессами. Для этого используются MPI функции MPI_Dims_create, MPI_Cart_create, MPI_Cart_coords, MPI_Cart_shift. На каждом узле, для вычисления значений функции на границе локальной расчетной области необходимы значения от соседних узлов. Коммуникации реализованы с помощью не блокирующих пересылок MPI_Isend, MPI_Irecv. Так, сначала запускаются коммуникации; вычисляются значения во внутренней области; затем ожидается окончание пересылок и вычисляются граничные значения. Такой подход может скрыть часть коммуникаций за вычислениями и уменьшить количество синхронизаций в сравнении с выполнением блокирующих пересылок.

Для выполнения гибридной версии программы MPI + OpenMP, к существующим циклам внутри MPI программы просто добавляются те же прагмы что и в исключительно OpenMP программе по той же логике.

## 2. Результаты расчётов

При проведении вычислительных экспериментов использовалась система IBM Polus.

В качестве входных параметров, программе передаётся размер вычислительной области (L), количество узлов сетки (N), число шагов по времени и размер шага (steps, dt), и количество OpenMP нитей для каждого процесса (threads). Для просмотра способа передачи параметров в программу можно воспользоваться флагом **help**. Если не задавать параметры явно они примут значения по умолчанию.

Для запусков использовались следующие параметры: размер домена 1.0 или PI, число узлов 128, 256 и 512, 20 шагов по времени при размере шага 1.e-5. Сравнение с аналитическим решением выполнялось после выполнения заданного количества шагов по времени и приведено в таблице с результатами.

Для компиляции использовался gcc 9.3.1 и соответственно mpicxx с OpenMPI 4.0.2. Использовались доступные флаги оптимизации:

*-O3 -ffast-math -funroll-loops -ftree-vectorize -mcpu=power8*

При проведении экспериментов все узлы запрашивались в эксклюзивное пользование (флаг для запуска **-x**). Всего выполнялось по 5 запусков, из которых выбиралось наименьшее время работы. Однако, некоторая неточность в получившимся времени работы остаётся, в силу отсутствия начальных разминочных итераций, время работы которых не входит в замеряемое время.

Первая таблица с результатами запусков последовательной версии программы. С ней сравниваются параллельные версии программы. Погрешность вычислений уменьшается пропорционально росту размера сетки.

| Число точек сетки | Время решения T, сек | Погрешность δ |
|---|---|---|
| 128 | 1.542844 | 1.19382E-09 |
| 256 | 10.854197 | 2.96158E-10 |
| 512 | 87.310481 | 7.37773E-11 |

Последовательная программа запускалась на фронтенд узле, поэтому время работы может оказаться выше чем при запуске на вычислительном узле с эксклюзивном доступом.

Для запусков OpenMP версии программы, использовался предоставленный скрипт по привязке нитей к ядрам: */polusfs/lsf/openmp/launchOpenMP.py*. В каждом запуске нити размещались на одном узле. Для 32 нитей использовались 16 ядер, чтобы исключить обмен данными между узлами. Для этого использовались флаги **-R «span[hosts=1] affinity[core(X)]»** и *OMP_NUM_THREADS=X*, а для 32 узлов — комбинация *affinity[core(16)]* и *OMP_NUM_THREADS=32*.

Также дополнительно использовались переменные окружения :

*export OMP_SCHEDULE=static*
*export OMP_PROC_BIND=true*
*export OMP_DYNAMIC=false*

Результаты запусков приведены во второй таблице:

| Число OpenMP нитей | Число точек сетки | Время решения T, сек | Ускорение S | Погрешность δ |
|---|---|---|---|---|
| 1 | 128 | 1.49196 | 1.03 | 1.19382E-09 |
| 2 | | 0.818046 | 1.89 | |
| 4 | | 0.371213 | 4.16 | |
| 8 | | 0.197853 | 7.80 | |
| 16 | | 0.10615 | 14.53 | |
| 32 | | 0.086688 | 17.80 | |
| 1 | 256 | 12.0124 | 0.90 | 2.96158E-10 |
| 2 | | 6.34441 | 1.71 | |
| 4 | | 2.8565 | 3.80 | |
| 8 | | 1.50534 | 7.21 | |
| 16 | | 0.789385 | 13.75 | |
| 32 | | 0.637678 | 17.02 | |
| 1 | 512 | 88.9711 | 0.98 | 7.37773E-11 |
| 2 | | 44.181 | 1.98 | |
| 4 | | 22.3871 | 3.90 | |
| 8 | | 11.7455 | 7.43 | |
| 16 | | 6.08428 | 14.35 | |
| 32 | | 4.88142 | 17.89 | |

Получаемое ускорение линейно растёт с увеличением числа нитей, вплоть до использования 16 нитей на одном узле. Эффективность ускорения при этом находится в районе 90%. При использовании 32 нитей эффективность ускорения снижается в связи с гипертредингом — запуском 2 нитей на одном ядре, что приводит к конкуренции за вычислительные ресурсы. Максимальное ускорение в итоге составляет порядка 17-18 раз, при использовании 32 нитей. Погрешность вычислений при этом не меняется.

Для запусков MPI версии программы, процессы поровну делились между вычислительными узлами для равномерного распределения нагрузки между используемыми каналами памяти. Для этого использовался флаг **-R «span[ptile=x]»**.

Ниже представлена таблица с результатами выполнения MPI версии программы.

| Число процессов MPI | Число точек сетки | Время решения T, сек | Ускорение S | Погрешность δ |
|---|---|---|---|---|
| 1 | 128 | 2.067163 | 0.75 | 1.19382E-09 |
| 2 | | 0.967529 | 1.59 | |
| 4 | | 0.497234 | 3.10 | |
| 8 | | 0.250571 | 6.16 | |
| 16 | | 0.134743 | 11.45 | |
| 32 | | 0.082457 | 18.71 | |
| 1 | 256 | 15.316475 | 0.71 | 2.96158E-10 |
| 2 | | 7.758737 | 1.40 | |
| 4 | | 3.957109 | 2.74 | |
| 8 | | 2.163573 | 5.02 | |
| 16 | | 1.163507 | 9.33 | |
| 32 | | 0.551702 | 19.67 | |
| 1 | 512 | 117.331015 | 0.74 | 7.37773E-11 |
| 2 | | 59.466812 | 1.47 | |
| 4 | | 30.351175 | 2.87 | |
| 8 | | 15.965505 | 5.47 | |
| 16 | | 8.517064 | 10.25 | |
| 32 | | 5.022387 | 17.38 | |

Погрешность вычислений совпадает с погрешностью последовательной программы. По результатам экспериментов получен линейный рост ускорения от числа используемых процессов, при этом эффективность ускорения не деградирует даже при использовании максимальных 32 процессов. Масштабируемость ускорения от числа процессов в чистом виде проявляется при сравнении с 1 MPI процессом, при котором время работы MPI программы примерно на 30% медленнее последовательной версии. Это связано с накладными расходами на коммуникации и использованием буферов, при котором никакого распараллеливания

нет. Соответственно, максимальное получаемое ускорение составляет порядка 18 раз при 32 процессах относительно последовательной версии программы, и до 27 раз относительно 1 MPI процесса. Высокая эффективность ускорения наблюдается уже на сетке с 128 узлами. В задаче объем вычислений зависит от размера сетки как $O(N^3)$, а объем коммуникаций как $O(N^2)$. Соответственно, задача хорошо масштабируется, а увеличение размера сетки не приводит к росту эффективности.

Гибридная версия программы была протестирована при использовании 4 нитей и нескольких MPI процессов. При этом каждому процессу выделялось по 2 ядра для размещения нитей, с помощью флага **-R «affinity[core(2)]»**. Результаты приведены в таблице ниже.

| Число процессов MPI | Число OpenMP нитей | Число точек сетки | Время решения T, сек | Ускорение S | Погрешность δ |
|---|---|---|---|---|---|
| 1 | 4 | 128 | 1.256695 | 1.23 | 1.19382E-09 |
| 2 | 4 | | 0.611654 | 2.52 | |
| 4 | 4 | | 0.333057 | 4.63 | |
| 8 | 4 | | 0.164191 | 9.39 | |
| 1 | 4 | 256 | 9.403059 | 1.15 | 2.96158E-10 |
| 2 | 4 | | 4.728261 | 2.29 | |
| 4 | 4 | | 2.441044 | 4.45 | |
| 8 | 4 | | 1.370065 | 7.92 | |
| 1 | 4 | 512 | 73.487619 | 1.19 | 7.37773E-11 |
| 2 | 4 | | 37.033055 | 2.36 | |
| 4 | 4 | | 18.920101 | 4.61 | |
| 8 | 4 | | 9.896461 | 8.82 | |

Получено ускорение в среднем в полтора раза в сравнении с MPI версией программы, использующей только 8 MPI процессов. Такой результат достигается за счёт использования по факту 16 ядер для размещения нитей на 8 MPI процессах, а не 8 ядер. При использовании числа ядер равному числу процессов вместе с гипертредингом для запуска большого числа нитей, гибридная программа работает медленнее чисто MPI или тем более чисто OpenMP версии.

Приведённые выше результаты получены для запуска программы с расчётной областью *1.0 x 1.0 x 1.0*. Помимо этого, также выполнялись тесты для расчётной области *Pi x Pi x Pi*, но время работы программы не зависит от этого параметра, поскольку не меняется количество вычислений в расчётной сетке. Поэтому отельные таблицы с временами выполнения для *L = Pi* не были включены. Однако, при изменении размера домена меняется погрешность вычислений. Ниже приведена таблица с погрешностью для обеих расчётных областей.

| Число точек сетки | Погрешность для L = 1.0 | Погрешность для L = Pi |
|---|---|---|
| 128 | 1.19382E-09 | 1.21085E-10 |
| 256 | 2.96158E-10 | 3.00353E-11 |
| 512 | 7.37773E-11 | 7.48345E-12 |

Погрешность для запусков с параметром *L = Pi* масштабируется также, как и для параметра *L = 1.0* что полностью соответствует ожидаемому результату.